CS 314 Principles of Programming Langauges
Project 1: A Compiler for the TinyL Language

THIS IS NOT A GROUP PROJECT! You may talk about the project and possible solutions in general terms, but must not share code. In this project, you will be asked to write a recursive LL(1) parser and code generator for the tinyL language. Your compiler will generate assembly-like code from a modified RISC instruction set. You will be provided with a virtual machine that can run your assembly programs.

1 Background

1.1 The tinyL language

tinyL is a simple expression language that allows assignments and basic I/O operations.

⟨program⟩	::=	⟨stmtlist⟩ !	
⟨stmtlist⟩	::=	⟨stmt⟩ ⟨morestmts⟩	
⟨morestmts⟩	::=	; ⟨stmtlist⟩ | ε	
⟨stmt⟩	::=	⟨assign⟩ | ⟨read⟩ | ⟨print⟩	
⟨assign⟩	::=	⟨variable⟩ = ⟨expr⟩	
⟨read⟩	::=	$ ⟨variable⟩	
⟨print⟩	::=	# ⟨variable⟩	
⟨expr⟩	::=	+ ⟨expr⟩ ⟨expr⟩ |	integer addition
		− ⟨expr⟩ ⟨expr⟩ |	integer subtraction
		* ⟨expr⟩ ⟨expr⟩ |	integer multiplication
		& ⟨expr⟩ ⟨expr⟩ |	bitwise and
		| ⟨expr⟩ ⟨expr⟩ |	bitwise or
		^ ⟨expr⟩ ⟨expr⟩ |	bitwise xor
		≤ ⟨expr⟩ ⟨expr⟩ |	bitwise left shift
		≥ ⟨expr⟩ ⟨expr⟩ |	bitwise right shift
		∼ ⟨expr⟩ |	bitwise not
		⟨variable⟩ |	
		⟨digit⟩	
⟨variable⟩	::=	a | b | c | d | e | f	
⟨digit⟩	::=	0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9	

The following are some examples of tinyL programs

```
a=^*89+47;#a!        Computes a = (8 * 9) ^ (4 + 7), then prints a.
$a;b=+a+25;%b!       Reads a, computes b = a + (2 + 5), then prints b.
```

1.2 Target Architecture

The target architecture is a simple RISC machine with unlimited registers. All variables and registers store a 64-bit signed integer. Arithmetic operations can only be applied to registers, while reading user input and printing must be done with variables. This means that doing computation with variables requires calling load and store for moving data to and from registers.

Below is the instruction set of the target architecture. Constants are written $\#\langle\mathsf{const}\rangle$, where $\langle\mathsf{const}\rangle$ is any signed 64-bit value (e.g. $\#20$, $\#{-}142$). $\langle\mathsf{id}\rangle$ is a variable a through f. R_x, R_y, and R_z represent any three registers, formatted rx, ry, rz for nonnegative numbers x, y, z. (e.g. $r1$, $r2$, $r3$). A program consists of multiple RISC instructions separated by newlines.

instruction format	description	semantics
	memory instructions	
`LOADI` R_x $\#\langle\mathsf{const}\rangle$	Load constant value $\#\langle\mathsf{const}\rangle$ into register R_x	$R_x \leftarrow \langle\mathsf{const}\rangle$
`LOAD` R_x $\langle\mathsf{id}\rangle$	Load the value of variable $\langle\mathsf{id}\rangle$ into register R_x	$R_x \leftarrow \langle\mathsf{id}\rangle$
`STORE` $\langle\mathsf{id}\rangle$ R_x	Store the value of register R_x into variable $\langle\mathsf{id}\rangle$	$\langle\mathsf{id}\rangle \leftarrow R_x$
	arithmetic instructions	
`ADD` R_x R_y R_z	Add contents of registers R_y and R_z, then store result in R_x	$R_x \leftarrow R_y + R_z$
`SUB` R_x R_y R_z	Subtract contents of register R_z from register R_y, then store result in R_x	$R_x \leftarrow R_y - R_z$
`MUL` R_x R_y R_z	Multiply contents of registers R_y and R_z, then store result in R_x	$R_x \leftarrow R_y * R_z$
	bitwise arithmetic instructions	
`AND` R_x R_y R_z	Take bitwise AND for contents of registers R_y and R_z, then store result in R_x	$R_x \leftarrow R_y \,\&\, R_z$
`OR` R_x R_y R_z	Take bitwise OR for contents of registers R_y and R_z, then store result in R_x	$R_x \leftarrow R_y \mid R_z$
`XOR` R_x R_y R_z	Take bitwise XOR for contents of registers R_y and R_z, then store result in R_x	$R_x \leftarrow R_y \wedge R_z$
`LS` R_x R_y R_z	Perform bitwise left shift for contents of register R_y by R_z, then store result in R_x	$R_x \leftarrow R_y \ll R_z$
`RS` R_x R_y R_z	Perform bitwise right shift for contents of register R_y by R_z, then store result in R_x	$R_x \leftarrow R_y \gg R_z$
`NOT` R_x R_y	Perform bitwise not of contents of register R_y, then store result in R_x	$R_x \leftarrow \sim R_y$
	I/O instructions	
`READ` $\langle\mathsf{id}\rangle$	Read value of variable $\langle\mathsf{id}\rangle$ from standard input	read($\langle\mathsf{id}\rangle$)
`WRITE` $\langle\mathsf{id}\rangle$	Write value of variable $\langle\mathsf{id}\rangle$ to standard input	print($\langle\mathsf{id}\rangle$)

2 Project Description

This project can be written in C/C++, Python, or Java. A partially implemented recursive LL(1) descent parser with code generation has been provided for all four languages. Instructions for compiling the starter implementations are detailed below. Additionally, a RISC interpreter and tinyL test cases are included in the start package.

2.1 Compiler

The compiler is given file paths for an input tinyL file and output RISC machine code file. It should read the tinyL file with a recursive descent parser. Any whitespaces (spaces, tabs, or newlines) should be ignored. During parsing, it should write machine code to the output file. The starter compiler can only generate code for ⟨digit⟩ and ⟨expr⟩ which only contain constants and the + operator. You will need to add code in the provided stubs to generate correct RISC machine code for the entire program. You should not have to change the signatures of the functions. Hint: the left-handed and right-handed occurrences of variables are treated differently.

You are not required to use the template or implement the recursive descent parser exactly as described. Other techniques are acceptable, as long as they produce correct output.

2.2 Build Steps

The autograder takes any code files for the supported languages. The makefile included in the starter package is used to build the project for C and C++. Below is a table with details on how the autograder will compile and run each language.

Language	**Entrypoint**	**Compilation and Running**
C	compiler.c	`make compiler` `./compiler input.tinyL program.out`
C++	Compiler.cpp	`make compiler` `./compiler input.tinyL program.out`
Python	compiler.py	`# python is not compiled` `python3 compiler.py input.tinyL program.out`
Java	Compiler.java	`javac Compiler.java` `java Compiler input.tinyL program.out`

2.3 Virtual Machine

The interpreter is a virtual machine that executes a RISC program, located at interpreter.py. To run the interpreter, type python3 interpreter.py risc.out.

- If `READ` ⟨id⟩ is called, the user is prompted for the value of ⟨id⟩ from stdin.
- If `WRITE` ⟨id⟩ is called, the current value of ⟨id⟩ is written to stdout.

The virtual machine provides an arbitrary number of registers. There are also **6 memory variables** named ('a', ... ,'f'). Note that I/O can only be performed on variables, while arithmetic operations can only be performed on registers.

As a side note, real hardware typically does not have infinite registers. Therefore compilers usually need to map virtual registers to physical ones in a process called *register allocation*. This is not implemented in this project.

3 Grading

3.1 Base Project

Depending on the language you choose, your submission should include the **entrypoint file** and any additional **imported source / header files**. The autograder will match the first programming language it sees and attempt to build it based on the arguments in Build Steps. The appropriate Makefile will automatically be used for C/C++ submissions. **We will not support custom Makefiles or libraries.**

Your program will be graded on functionality. This will be verified through automatic testing on 20 syntactically correct test cases. No error handling is required. 16 of these test cases will be provided in the starter package. The remaining four will be hidden.

3.2 Extra Credit

You can use the algorithms discussed in class to implement the code generator and will receive full credit for each test case if the generated RISC code runs correctly. However, the number of instructions generated by this method can be significantly improved. Below are five different possible optimizations.

- **Constant Propogation** If an ⟨expr⟩ only consists of constants, we can precompute the expression and subistute it with the resulting value. Furthermore, a ⟨variable⟩ which is known to be constant can be replaced with said constant.

  a=-+452;b=+ab! can be rewritten as a=7;b=+b7!

- **Common sub-expression elimination (CSE)** If there are several instances of an indentical expression, these can be stored to a single variable or register to eliminate redundant computation.

  x=+4*ab;y=+5*ab! can be rewritten as c=*ab;x=+4c;y=+5c!

- **Dead code elimination** Parts of the program which do not affect input and output behavior are called "dead code". These parts of the program can simply be removed to save computation.

  a=1;b=2;#a! can be rewritten as a=1;#a!

- **Algebraic simplification** Some arithmetic operations with variables can be simplified with identites. This also includes bitwise operations.

  a=^b-cc! can be rewritten as a=^b0!, then a=b!

- **Iterative optimization** The above optimizations can be run in conjunction. Rerunning these techniques to a fixed point can yield better instruction count.

  $a;b=-aa;a=+ab;#a! apply algebraic simplification
  ⇒ $a;b=0;a=+ab;#a! apply constant propogation
  ⇒ $a;b=0;a=+a0;#a! apply algebraic simplification
  ⇒ $a;b=0;a=a;#a! apply dead code elimination
  ⇒ $a;#a!

Each of the first four optimizations will have its own additional test case. Hatsune Miku has provided the autograder with an implementation for each of these optimizations. You will receive extra credit based on how your submission performs in comparison.

The optimization is considered completed if the submitted compiler produces a working program that saves at least 75% of the instructions Hatsune Miku's implementation does. Successfully completing each individual optimization is worth 1% extra credit on this project. Finally, cumulatively saving at least 97% of the instructions Hatsune Miku's optimizer does will give an additional 1% extra credit.

Note that there is no one "correct" way to implement these optimizations. It is possible to perform these steps with preprocessing, postprocessing, or even performing transformations on an abstract syntax tree.

A word of caution: these optimizations should be done with the requirement that all operations are meant to be performed on 64-bit signed integers. A wrapper class that emulates this behavior for python programs is located in the interpreter. The autograder compiles the C/C++ programs with -fwrapv. This is automatically handled in Java long.

4 Starter Package

The starter code package includes partial implementations of an LL(1) parser in the respective language. There is also an interpreter called interpreter.py. Finally, 16 test cases will be provided under tests/.

5 Questions

Any questions should be posted on Piazza. Good luck!